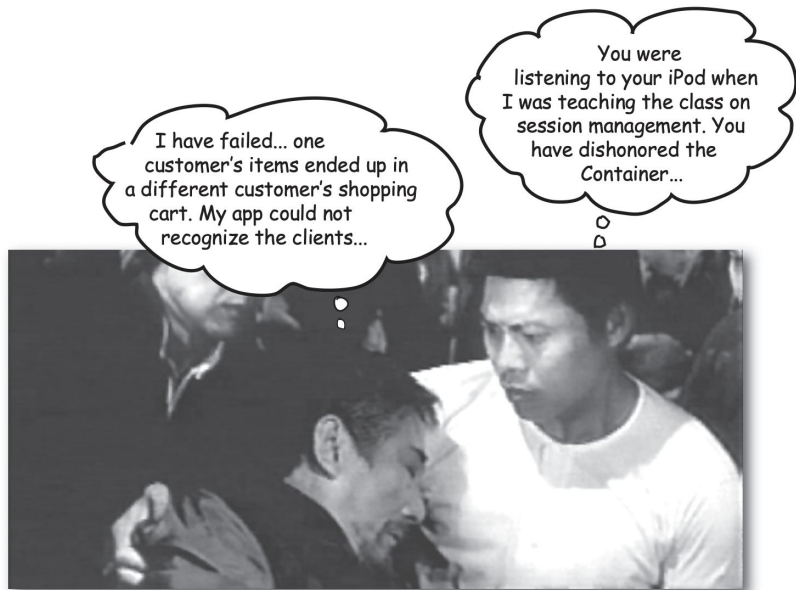


Cookies e Sessões em Aplicações Web

Gerenciamento de estado e implementação com Jakarta EE e Servlets



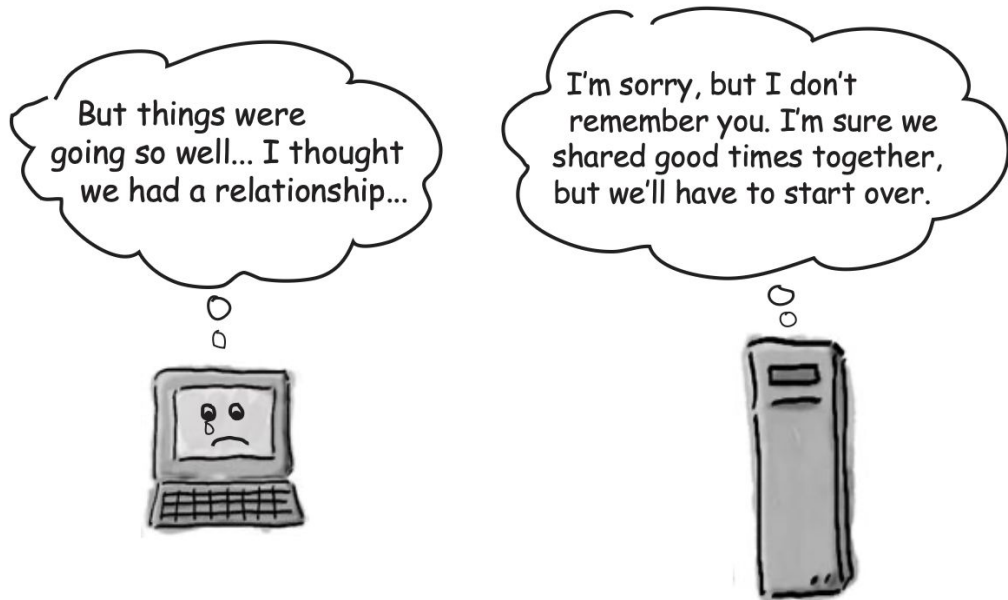
Objetivos da Aula

Ao final desta aula, espera-se compreender como manter um estado de conversação entre cliente e servidor usando cookies e gerenciamento de sessões, implementando esses conceitos na prática com Servlets Java.

Competência	Resultado esperado
Objetos de Sessão	Escrever código em Servlets capaz de armazenar dados em um objeto de sessão e recuperar esses dados em novas requisições do cliente.
Gerenciamento de Sessão	Criar e destruir objetos de sessão quando apropriado, usando a API HttpSession.
Cookies	Usar cookies para gerenciamento de sessão e para troca de pequenos dados entre cliente e servidor.

O problema começa no HTTP sem estado

O protocolo HTTP processa mensagens independentes; a aplicação precisa criar continuidade.



Sem memória nativa

O servidor não sabe, por HTTP puro, quem fez a requisição anterior.

Estado explícito

Cookies e sessões associam requisições sucessivas ao mesmo fluxo.

Estado torna a web utilizável

A continuidade transforma páginas independentes em uma experiência coerente.

Autenticação

Manter o usuário reconhecido após o login, sem solicitar credenciais a cada requisição.

Personalização

Aplicar idioma, tema ou preferências sem reconfigurar a interface constantemente.

Carrinho

Acumular itens escolhidos em páginas diferentes antes da finalização da compra.

Fluxo

Conduzir formulários em etapas, mantendo dados temporários e decisões intermediárias.

HTTP Cookies

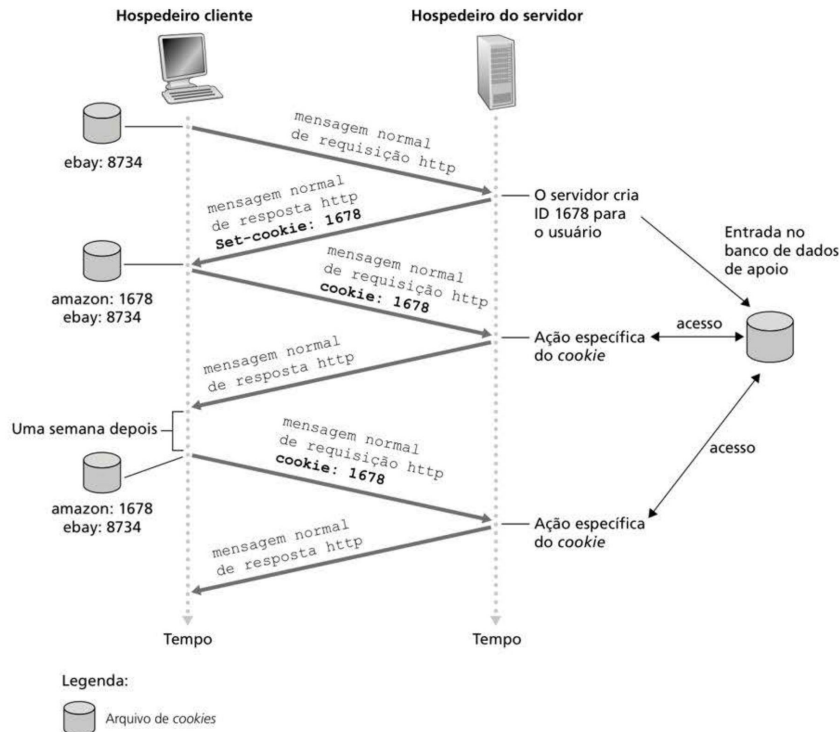
Um cookie é um par nome-valor controlado por cabeçalhos HTTP.

DEFINIÇÃO

Cookie é um pequeno fragmento de dados configurados por um servidor HTTP e enviado ao cliente.

A especificação de cookies, (RFC 6265), padroniza esse mecanismo por meio dos cabeçalhos **Set-Cookie** e **Cookie**.

Os Cookies permitem que sites monitorem seus usuários, o que é amplamente praticado hoje em dia.



HTTP Cookies

1. Servidor responde



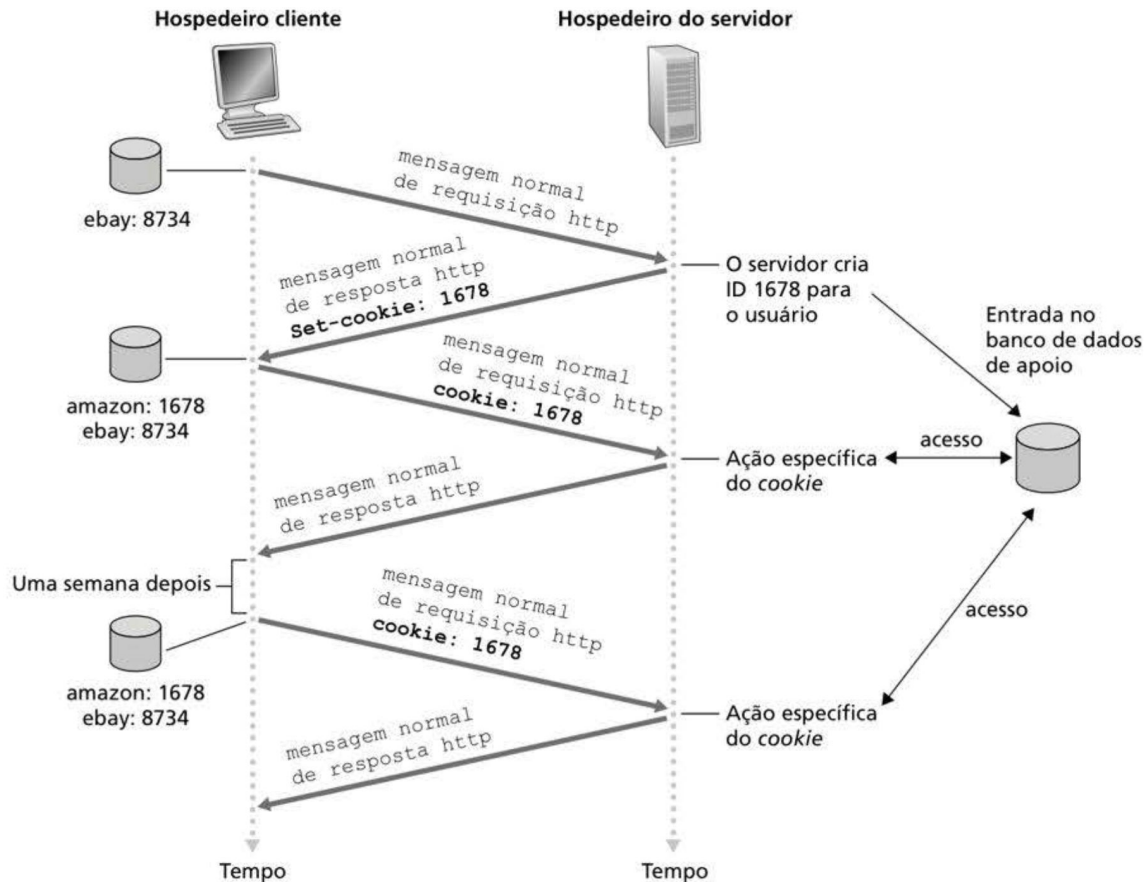
2. Navegador Armazena



3. Cliente Requisita



4. Servidor Recupera Cookie



HTTP Cookies

1. Servidor responde



2. Navegador Armazena



3. Cliente Requisita



4. Servidor Recupera Cookie

Cookies



"Set-Cookie" is just another header sent in the response.



```
HTTP/1.1 200 OK
Set-Cookie: JSESSIONID=0AAB6C8DE415
Content-Type: text/html
Content-Length: 397
Date: Wed, 19 Nov 2003 03:25:40 GMT
Server: Apache-Coyote/1.1
Connection: close

<html>
...
</html>
```

HTTP Response

Here's your cookie with the session ID inside...



OK, here's the cookie with my request



```
POST /select/selectBeerTaste2.do HTTP/1.1
Host: www.wickedlysmart.com
User-Agent: Mozilla/5.0
Cookie: JSESSIONID=0AAB6C8DE415
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
```

HTTP Request



HTTP Cookies

Atributos determinam onde, quando e por quanto tempo o navegador envia o cookie.

Domain	Define o domínio para o qual o cookie pode ser enviado. Quando omitido, vale para o host que o criou.
Path	Restringe caminhos dentro do site. Um cookie de /app não precisa ser enviado para todo o site.

HTTP Cookies

Atributos determinam onde, quando e por quanto tempo o navegador envia o cookie.

Expires	Marca uma data absoluta de expiração. Depois dela, o navegador deve descartar o cookie.
Max-Age	Define duração em segundos. O valor 0 instrui o navegador a excluir o cookie.

Cookie não é sessão

Cookie

mecanismo no cliente

Local: armazenado e reenviado pelo navegador.

Conteúdo: pequeno par nome-valor com atributos.

Controle: pode ser removido pelo usuário e é enviado em requisições elegíveis.

Risco: não deve guardar dados sensíveis em claro.

Sessão

estado no servidor

Local: mantida pelo contêiner ou aplicação.

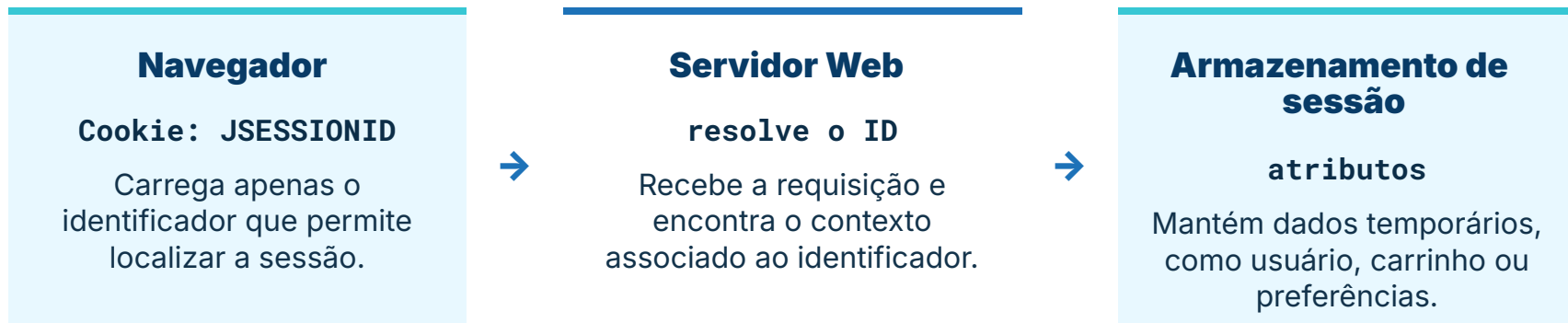
Conteúdo: atributos temporários associados ao usuário.

Controle: expiração e invalidação são decisões do servidor.

Risco: depende da proteção do identificador de sessão.

Sessões

A sessão associa várias requisições HTTP a um contexto temporário controlado pela aplicação.



Vantagem

Dados sensíveis não precisam ser enviados integralmente ao cliente em cada requisição.

Responsabilidade

A aplicação precisa controlar expiração, renovação, invalidação e consumo de memória.

Sessões: aspectos de segurança

Quem controla o identificador pode tentar assumir a continuidade da interação com a aplicação.

Captura

Exposição em rede insegura, logs, URLs ou scripts pode revelar o identificador.

confidencialidade

Fixação

Um atacante induz a vítima a usar um ID conhecido antes da autenticação.

renovar no login

Reutilização

Sessões muito longas ampliam a janela de ataque caso o ID seja comprometido.

expiração

Controles essenciais

HTTPS: reduz exposição em trânsito.

HttpOnly: dificulta acesso por JavaScript.

Secure: envia apenas em conexões seguras.

Renovação: troca a sessão em eventos sensíveis.

Sessões: aspectos de segurança

Atributos reduzem exposição, envio indevido e acesso por código no navegador.

Atributo	Finalidade	Efeito esperado
Secure	Restringe envio a HTTPS.	Evita transmissão do cookie em conexões não protegidas.
HttpOnly	Bloqueia acesso por JavaScript.	Reduz impacto de ataques de script sobre cookies de sessão.
SameSite	Controla envio em navegação entre sites.	Ajuda a reduzir riscos associados a CSRF.

Cabeçalho HTTP ilustrativo

```
Set-Cookie: JSESSIONID=abc123; Path=/app; Secure; HttpOnly; SameSite=Lax
```

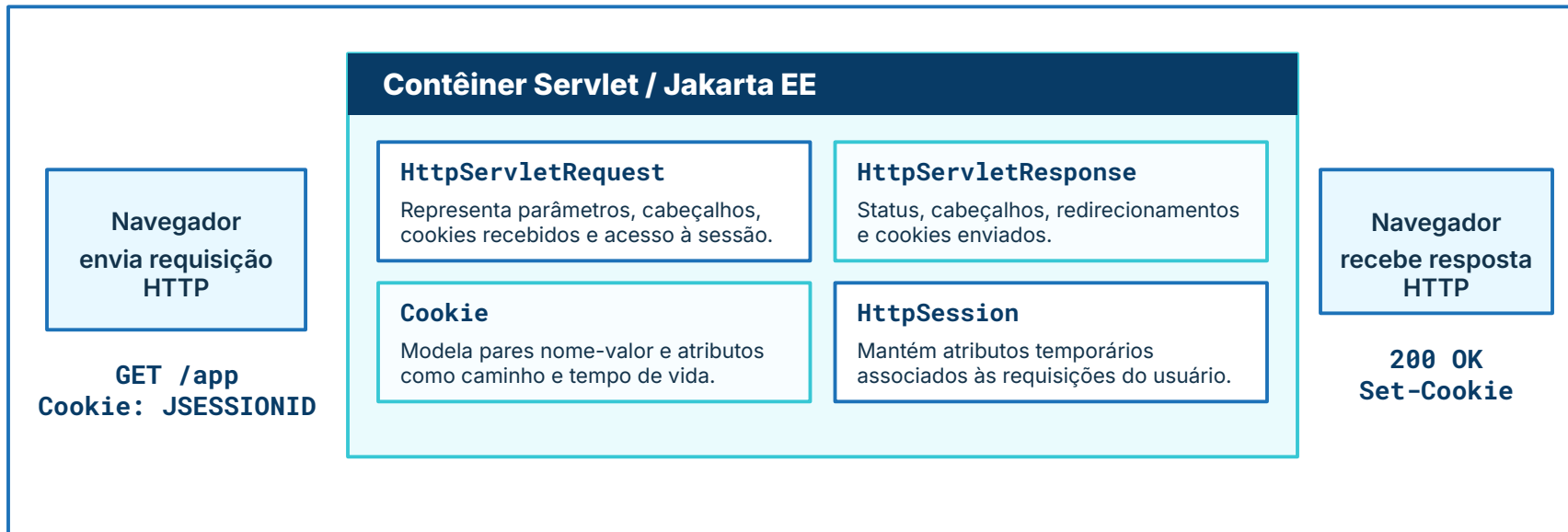
Cookies e Sessões: implementação

Antes de programar, é preciso escolher onde cada tipo de estado deve viver.

DECISÃO	Cookie	Sessão	Armazenamento local	Banco de dados
Onde fica?	No navegador e enviado automaticamente em requisições elegíveis.	No servidor, associado a um identificador de sessão.	No navegador, acessível pela aplicação cliente.	No servidor ou serviço persistente de dados.
Uso adequado	Preferências simples ou transporte de ID opaco.	Login ativo, carrinho temporário e fluxo multipágina.	Preferências de interface no cliente.	Dados permanentes, históricos, auditoria e regras de negócio.
Risco principal	Exposição em cada requisição.	Sequestro ou fixação do identificador; CSRF.	Pode ser lido por JavaScript e não deve guardar segredo.	Persistência indevida ou controle fraco de autorização.
Quando usar?	Quando o servidor precisa receber dados pequenos.	Quando o estado é temporário e deve ficar sob controle do servidor.	Quando o dado é local à interface e não é confidencial.	Quando o dado precisa sobreviver a expiração da sessão.

Cookies e Sessões: implementação

A API Servlet materializa cookies, requisições, respostas e sessões em objetos Java.



Cookies e Sessões: implementação

Um cookie é criado no servidor e enviado ao navegador como parte da resposta HTTP.

Sequência segura

1. Defina nome e valor simples; evite dados sensíveis em claro.
2. Restrinja o escopo com caminho e tempo de vida.
3. Use atributos de proteção quando disponíveis pela API.
4. Envie o cookie com `response.addCookie`

Exemplo: Cookie de preferência

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws IOException {
    Cookie tema = new Cookie("tema", "escuro");
    tema.setPath(request.getContextPath());
    tema.setMaxAge(60 * 60 * 24 * 30); // 30 dias
    tema.setHttpOnly(true);

    response.addCookie(tema);
    response.sendRedirect("preferencias.jsp");
}
```

Atenção!

Em aplicações reais, cookies usados em autenticação ou sessão também devem ser transmitidos por HTTPS e configurados com atributos compatíveis com a política de segurança do sistema.

Cookies e Sessões: implementação

Cookies chegam como parte da requisição HTTP e precisam ser tratados como entrada do cliente.

Fluxo de leitura

`request.getCookies()`
pode retornar **null**

Percorra o vetor e compare o nome do cookie.

Valide o valor antes de usá-lo em decisão de negócio.

Exemplo: recuperar uma preferência

```
private String lerCookie(HttpServletRequest request,
                        String nome) {
    Cookie[] cookies = request.getCookies();
    if (cookies == null) {
        return null;
    }

    for (Cookie c : cookies) {
        if (nome.equals(c.getName())) {
            return c.getValue();
        }
    }
    return null;
}
```

Cuidado!

O valor do cookie vem do navegador. Por isso, a aplicação não deve confiar nele como prova de identidade, autorização ou integridade sem validações adicionais no servidor.

Cookies e Sessões: implementação

O servidor não apaga diretamente o armazenamento do navegador; ele envia uma instrução de expiração.

Regra operacional

**mesmo nome + mesmo escopo
+ Max-Age = 0**

Para que o navegador reconheça qual cookie deve expirar, o nome, o domínio e o caminho devem corresponder ao cookie criado anteriormente.

Exemplo: remover cookie de preferência

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws IOException {
    Cookie tema = new Cookie("tema", "");
    tema.setPath(request.getContextPath());
    tema.setMaxAge(0);           // expira agora
    tema.setHttpOnly(true);

    response.addCookie(tema);
    response.sendRedirect("preferencias.jsp");
}
```

Cookies e Sessões: implementação

A API de sessão permite associar atributos temporários a um navegador ou usuário ao longo de múltiplas requisições.

Operações principais

`getSession()` obtém ou cria uma sessão.

`getSession(false)` consulta sem criar.

`setAttribute()` grava dados temporários.

`getAttribute()` recupera atributos.

`removeAttribute()` remove um item.

`invalidate()` encerra a sessão.

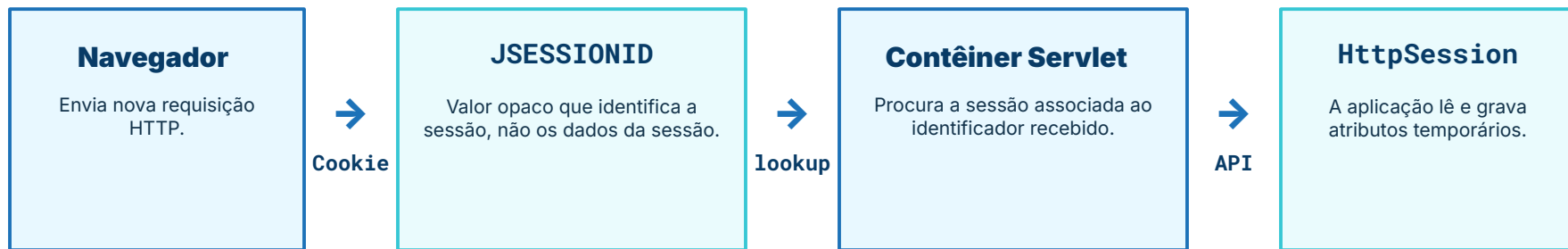
Exemplo: armazenar usuário autenticado

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws IOException {
    Usuario usuario = autenticar(request);

    if (usuario != null) {
        HttpSession sessao = request.getSession();
        sessao.setAttribute("usuario", usuario.getId());
        response.sendRedirect("/area-restrita");
    } else {
        response.sendRedirect("/login?erro=1");
    }
}
```

Cookies e Sessões: implementação

O identificador de sessão é a ponte entre requisições do navegador e estado mantido pelo contêiner.



Valor opaco

O identificador deve ser imprevisível e não conter informações de negócio.

Associação server-side

Os atributos ficam no servidor; o navegador normalmente transporta só a chave.

Proteção necessária

Se alguém obtém o identificador, pode tentar assumir a sessão enquanto ela estiver válida.

Cookies e Sessões: implementação

Após autenticar, a aplicação deve reduzir o risco de fixação de sessão.

1. Sessão anterior

Pode ter sido criada antes do login.



2. Autenticação

Credenciais são validadas no servidor.



3. Nova sessão

O usuário autenticado é associado a uma sessão renovada.

Exemplo: renovar antes de gravar identidade

```
Usuario usuario = autenticar(request);
if (usuario != null) {
    HttpSession antiga = request.getSession(false);
    if (antiga != null) {
        antiga.invalidate();
    }
    HttpSession nova = request.getSession(true);
    nova.setAttribute("usuarioId", usuario.getId());
    response.sendRedirect("/area-restrita");
} else {
    response.sendRedirect("/login?erro=1");
}
```

Por que renovar?

A renovação impede que um identificador de sessão conhecido antes do login continue válido depois da autenticação.

Guarde na sessão apenas dados mínimos, como o identificador interno do usuário.

Cookies e Sessões: implementação

Encerrar a interface não basta; o estado autenticado precisa ser encerrado no servidor.

Localizar sessão

Use `getSession(false)` para evitar criar sessão durante o logout.

Invalidar estado

`invalidate()` remove atributos e encerra a associação.

Redirecionar

Envie o usuário para login ou página pública após o encerramento.

Exemplo: Servlet de logout

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws IOException {
    HttpSession sessao = request.getSession(false);
    if (sessao != null) {
        sessao.invalidate();
    }
    response.setHeader("Cache-Control", "no-store");
    response.sendRedirect(request.getContextPath() + "/login");
}
```

Regra prática

O logout deve acontecer por ação explícita no servidor e não apenas por ocultar componentes da interface.

Depois do logout, áreas restritas devem revalidar a sessão antes de exibir conteúdo.

Cookies e Sessões: implementação

Nem toda sessão começa autenticada; ela também pode apoiar fluxos temporários antes do login.



Exemplo: guardar carrinho na sessão

```
HttpSession sessao = request.getSession();
Carrinho carrinho = (Carrinho) sessao.getAttribute("carrinho");

if (carrinho == null) {
    carrinho = new Carrinho();
    sessao.setAttribute("carrinho", carrinho);
}

carrinho.adicionar(produtoId, quantidade);
```

Decisão de projeto

Antes do login, a sessão pode manter estado transitório. Depois do login, a aplicação decide se transfere esse estado para o banco de dados.

O carrinho não deve substituir regras de preço e estoque validadas no servidor.

Cookies e Sessões: implementação

A aplicação deve declarar limites de inatividade e propriedades do cookie usado pela sessão.

Timeout

Define por quanto tempo uma sessão pode ficar inativa antes de expirar.

Cookie de sessão

Controla nome, caminho e atributos de proteção do identificador.

Política coerente

Tempo, autenticação e perfil de risco devem ser configurados em conjunto.

Exemplo em web.xml

```
<session-config>
  <session-timeout>30</session-timeout>

  <cookie-config>
    <http-only>true</http-only>
    <secure>true</secure>
  </cookie-config>
</session-config>
```

Exemplo programático

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws IOException {
    HttpSession sessao = request.getSession();

    sessao.setMaxInactiveInterval(30 * 60);

    response.sendRedirect("/home");
}
```